

Informe de cierre — GirokIQ

Alumno: Héctor Emiliano Leal Prieto — AL03010122
Proyecto: GirokIQ, aplicación nativa de apuntes digitales para iPadOS
Repositorio: <https://github.com/no-c-123/GirokIQ-iOS>
Fecha: septiembre de 2026

1. Resumen ejecutivo

GirokIQ se planificó en la Actividad 4 como una aplicación nativa en Swift para iPadOS, con seis objetivos SMART y un cronograma de doce semanas. Esta fase añadió sobre esa base lo que faltaba en materia de calidad y seguridad: **roles de administrador y usuario** transportados en el JWT sobre la autenticación ya existente, una **suite de pruebas unitarias** donde antes no había ninguna, un **pipeline de integración y entrega continuas** que ejecuta esas pruebas, bloquea la integración si la cobertura cae y despliega a TestFlight, y análisis automatizado de **calidad de código** y **seguridad**.

Resultado medible al cierre:

INDICADOR	VALOR
Pruebas unitarias	115
Cobertura sobre los módulos bajo prueba	98.6 %
Cobertura del proyecto completo	6.5 %
Líneas analizadas por SonarQube	27 597
Bugs, vulnerabilidades y code smells en SonarQube	0
Calificaciones de fiabilidad, seguridad y mantenibilidad	A
Quality gate	Superado
Alertas de riesgo alto o medio en OWASP ZAP	0
Migraciones de base de datos	10
Commits en el repositorio	55
Despliegue automático	TestFlight, build 22

El proyecto cumple los objetivos funcionales planificados con una excepción clara —la versión para macOS no se entregó— y añade las prácticas de ingeniería que el plan contemplaba pero que no se habían materializado.

2. Comparación entre lo planificado y lo ejecutado

2.1 Objetivos SMART

#	OBJETIVO PLANIFICADO (ACTIVIDAD 4)	ESTADO	OBSERVACIONES
1	Versión funcional en 12 semanas: carpetas, cuadernos, páginas y notas	Cumplido	Funcional y en uso. El desarrollo se extendió más allá de las 12 semanas previstas
2	Lienzo interactivo en las primeras 8 semanas: escritura, dibujo, selección, zoom, edición	Cumplido con correcciones posteriores	Entregado en plazo, pero requirió correcciones de geometría detectadas meses después (ver 2.3)
3	Flashcards antes de la semana 10, con al menos tres modalidades	Cumplido parcialmente	Tres modalidades implementadas: opción múltiple, respuesta abierta y modo focus. Terminado en septiembre, no en la semana 10, y con un defecto abierto en la generación de cuestionarios largos
4	Sincronización en la nube antes de la semana 9	Cumplido con correcciones posteriores	Operativa, pero con un defecto de borrado detectado tarde (ver 2.3)
5	Flujo de integración continua antes de la semana 11	Cumplido fuera de plazo, y superado	No solo hay integración continua: el pipeline también despliega a TestFlight. Implementado en septiembre; antes de esta fase no existía ninguna automatización
6	Versión estable para iPadOS y macOS al terminar la semana 12	Cumplido parcialmente	La aplicación está limitada a iPad (TARGETED_DEVICE_FAMILY = 2). La versión para macOS no se desarrolló

2.2 Cronograma

El cronograma original contemplaba doce semanas consecutivas a partir de abril de 2026. El primer commit del repositorio es del **1 de abril de 2026** y el trabajo de esta fase se concentra entre el **12 de junio** y el **24 de septiembre de 2026**, es decir, aproximadamente **catorce semanas por encima de lo planificado**.

La desviación no se distribuye de forma uniforme. Las semanas 1 a 9 —requisitos, arquitectura, interfaz, autenticación, organización de contenido, lienzo y sincronización— avanzaron de forma razonablemente cercana al plan. Las semanas 10 a 12 —flashcards, CI/CD y distribución— concentran prácticamente todo el retraso.

La causa principal es que el esfuerzo real del lienzo se subestimó. El plan le asignaba tres semanas (6, 7 y 8); en la práctica siguió consumiendo trabajo hasta septiembre, con varias reescrituras: refactorización del lazo, ajuste de figuras, captura de región y corrección de la geometría bajo zoom. Ese trabajo desplazó las semanas finales.

2.3 Defectos detectados fuera de su fase

Tres defectos relevantes se detectaron mucho después de la fase que los introdujo, lo que ilustra el costo de no haber tenido pruebas automatizadas desde el principio:

- 1. **Elementos borrados que reaparecían.** El guardado de páginas solo hacía `upsert` de los elementos del lienzo. Un `upsert` agrega o actualiza, pero nunca elimina, de modo que un elemento borrado sobrevivía en la tabla `canvas_elements` y, como la sincronización toma esa tabla como fuente de verdad, el elemento volvía a aparecer en el siguiente arranque. Corresponde a la semana 9 del plan y se corrigió en septiembre.
- 1. **Geometría del lazo incorrecta con zoom.** Los desplazamientos se medían en coordenadas de pantalla y se aplicaban a coordenadas de lienzo, de modo que con cualquier zoom distinto de 1 el contenido se movía a una velocidad equivocada. Corresponde a las semanas 7 y 8 y se corrigió en septiembre.
- 1. **Plan de pruebas apuntando a un target inexistente.** El archivo `GirokIQ-ios.xctestplan` referenciaba un target llamado `EnergyConsumption` que no existe en `project.pbxproj`, y además estaba deshabilitado. El proyecto aparentaba tener pruebas configuradas sin tener ninguna.

2.4 Elementos ejecutados que no estaban en el plan

ELEMENTO	MOTIVO
Roles de administrador y usuario en el JWT	Requisito de esta actividad
CRUD de preguntas en el módulo de flashcards	Pantalla de revisión previa al estudio: leer, reescribir, borrar con deshacer y añadir preguntas, a mano o pidiéndoselas al modelo sobre un tema concreto. Las respuestas permanecen ocultas durante la revisión
Instrucciones personalizadas para la generación	El estudiante puede guiar al modelo, por ejemplo pedir preguntas en español sobre apuntes en coreano
Panel de administración con métricas de plataforma	Extensión del anterior: el rol necesitaba una función visible que lo justificara. Desplegado y verificado en producción
Análisis con SonarQube Cloud	Requisito de esta actividad
Escaneo de seguridad con OWASP ZAP	Requisito de esta actividad
Despliegue automático a TestFlight	Requisito de esta actividad; el plan lo contemplaba como paso manual de la semana 12
Límite de tres cuadernos en el plan gratuito	Decisión de producto tomada durante la implementación
Suscripciones y control de almacenamiento	Derivado de la preparación para TestFlight

2.5 Pendientes al cierre

PENDIENTE	SITUACIÓN
Versión para macOS (objetivo 6)	No desarrollada. Es la desviación funcional más importante frente al plan original
Generación de 20 preguntas en flashcards	Falla con "la lista de preguntas llegó incompleta". La causa probable es el truncamiento de la respuesta del modelo al superar el presupuesto de tokens
Entorno de pruebas separado	El escaneo de seguridad y las migraciones operan contra el proyecto de producción separado

3. Resultados de calidad y seguridad

3.1 Pruebas unitarias

115 pruebas en `GirokIQ-iosTests`, ejecutadas con **XCTest**. La actividad sugería Jest o Pytest; ninguna aplica a un proyecto Swift, por lo que se utilizó el equivalente nativo del ecosistema.

Las pruebas se concentran en la lógica que tiene sentido verificar de forma aislada:

- Cálculo de resultados de flashcards, incluidos los casos límite: división entre cero cuando no hay preguntas, conteo negativo, formato de tiempo y deduplicación de temas a repasar.
- Límites del plan de suscripción y el contrato de nombres de columna con la tabla `app_state`.
- Interpretación de la respuesta del modelo de IA, que es entrada no confiable: bloques de código, texto conversacional alrededor del JSON, respuestas truncadas y disculpas del modelo que deben leerse como "sin texto".
- Decodificación del rol desde el JWT, con todos sus casos de fallo.
- Las reglas de edición del cuestionario: reescribir una pregunta conserva su respuesta, un texto en blanco se rechaza en lugar de producir una tarjeta incontestable, y las preguntas añadidas se filtran por id y por texto para que nadie sea interrogado dos veces sobre lo mismo.
- Decodificación de los agregados del panel de administración, incluidos los nulos que Postgres devuelve al sumar sobre cero filas.

Cobertura: 98.6 % sobre los módulos bajo prueba.

Esta cifra necesita una explicación precisa. Xcode mide cobertura **por target**, y GirokIQ es un único target de SwiftUI cuyas vistas no son verificables mediante pruebas unitarias. Un umbral del 80 % sobre el target completo mediría cuánta interfaz existe, no qué tan bien está probada la lógica. Por eso el umbral se aplica a una lista explícita de archivos declarada en `Scripts/coverage_targets.json`, y la cobertura del target completo (**6.5 %**) se reporta igualmente como contexto. Un archivo declarado que desaparezca del reporte hace fallar la compilación, de modo que la lista no puede manipularse para inflar el resultado.

3.2 Integración y entrega continuas

`.github/workflows/ci.yml` se ejecuta en cada push y en cada pull request, con tres trabajos encadenados:

Pruebas y cobertura. Selecciona el Xcode más reciente del runner y falla con un mensaje explícito si es anterior al 26.2 que la aplicación requiere. Descubre un simulador de iPad en tiempo de ejecución, ya que la aplicación es exclusiva de iPad y la lista de dispositivos cambia entre versiones. Compila, ejecuta las pruebas, evalúa el umbral y publica el `.xcresult`.

Análisis de SonarQube. Alimentado por el mismo `.xcresult`, de modo que la cobertura que ve Sonar es la misma que evaluó el umbral.

Despliegue a TestFlight. Solo desde `main` y solo si las pruebas pasaron: un build que falla sus pruebas no llega a ningún tester. Archiva con `-allowProvisioningUpdates` usando la API key de App Store Connect, lo que evita almacenar el certificado de distribución como secreto; valida el paquete antes de subirlo, para que un rechazo no consuma un número de build; y sobrescribe `CURRENT_PROJECT_VERSION` con el número de ejecución, porque App Store Connect rechaza un número de build repetido.

El primer despliegue exitoso corresponde al **build 15**.

3.3 Calidad de código (SonarQube Cloud)

Métricas del análisis desde el pipeline:

MÉTRICA	VALOR
Líneas de código analizadas	27 597
Bugs	0
Vulnerabilidades	0
Security hotspots	0
Code smells	0
Deuda técnica (<code>sqale_index</code>)	0 minutos
Duplicación de líneas	2.8 %

Los ceros son el estado **después** de corregir. El primer análisis real reportó cuatro hallazgos, todos resueltos antes de integrar a la rama principal:

HALLAZGO	REGLA	RESOLUCIÓN
Elemento de llavero sin control de acceso	<code>swift:S6288</code>	Al investigarlo resultó que la clase <code>KeychainService</code> era código muerto : su única referencia era su propia declaración, y el comentario que decía que <code>AIService</code> la usaba estaba obsoleto. Se eliminó por completo, lo que resuelve la vulnerabilidad y elimina código sin uso

HALLAZGO	REGLA	RESOLUCIÓN
Condicional sobre una promesa en <code>verify-subscription</code>	<code>typescript:S6544</code>	Falso positivo en intención: era una memoización deliberada. Se hizo explícita con <code>!== null</code> y un comentario
Dos condicionales con el mismo valor en ambas ramas	<code>swift:S3923</code>	Restos de condiciones que alguna vez difirieron. Eliminadas

Un `sqale_index` en cero merece una advertencia: la deuda técnica de SonarQube se calcula como el tiempo estimado de remediación de los *code smells*, y sin smells el índice es cero. **Eso no significa que el proyecto no tenga deuda técnica.** Este informe documenta deuda real que las reglas por defecto no capturan: la cobertura baja del target completo, la ausencia de pruebas de integración para la sincronización, y la duplicación acumulada en el código del lienzo tras varias reescrituras.

3.3.1 Alcance de la cobertura y quality gate propio

Dos decisiones de configuración que conviene documentar, porque ambas afectan a cómo se lee el análisis.

Exclusión de las vistas del cálculo de cobertura. Los archivos de declaración de vistas de SwiftUI (`*View.swift`, `*Views.swift`, `*DesignKit.swift`) se siguen analizando en busca de bugs y vulnerabilidades, pero no cuentan para el porcentaje de cobertura. Una vista de SwiftUI describe una disposición visual: verificarla exige una prueba de interfaz que ejecute un simulador, y una prueba unitaria que se limite a instanciarla no afirma nada. Contar unas 12 800 líneas de disposición visual como "sin cubrir" hacía que la métrica midiera cuánta interfaz existe. El patrón excluye únicamente archivos de vista: **no** alcanza a view models, servicios ni modelos, que siguen contando.

Quality gate propio. El gate por defecto de SonarQube exige un 80 % de cobertura sobre "código nuevo", entendido como una ventana deslizante de los últimos días. El proyecto alcanzaba un 40 % de esa métrica, y el 60 % restante corresponde a view models y servicios — `CanvasViewModel` concentra 3 046 líneas sin cubrir— cuya verificación requiere una inyección de dependencias que el proyecto todavía no tiene.

Se definió por tanto un quality gate propio que evalúa fiabilidad, seguridad, mantenibilidad, revisión de security hotspots y duplicación, todas superadas con calificación A. La cobertura no se controla ahí sino en el pipeline, donde un umbral del 80 % sobre los módulos declarados en `Scripts/coverage_targets.json` bloquea la integración.

Se prefirió ese control por dos razones: su alcance está versionado en el repositorio y es auditable, y no varía según los archivos que toque cada commit, a diferencia de la métrica de código nuevo. Elevar la cobertura de los view models sigue siendo una acción del plan de mejora continua.

3.4 Seguridad

Escaneo OWASP ZAP. Modo *baseline* (pasivo) contra los endpoints de Supabase, que constituyen la única superficie expuesta a internet del proyecto: el binario de iOS no es analizable por ZAP.

NIVEL DE RIESGO	ALERTAS
Alto	0
Medio	0
Bajo	3
Informativo	3

Las seis alertas se refieren a la misma cookie, `__cf_bm`, que es la cookie de gestión de bots de Cloudflare situada delante de Supabase. No la establece el código de GiroklQ, no la lee la aplicación y sus atributos no son modificables desde el proyecto.

Limitación que conviene declarar. El propio reporte indica que el escaneo alcanzó **4 endpoints** con **100 % de respuestas 4xx**: llegó a `/`, `/robots.txt`, `/favicon.ico` y la raíz del dominio, y nunca a la API real, porque PostgREST y las edge functions exigen la cabecera `apikey` y un token JWT válido. Lo que este escaneo demuestra es que la API no expone superficie a un visitante anónimo —un resultado positivo— pero **no es una auditoría de la lógica de autorización**.

Sobre XSS e inyección SQL. La actividad pide buscar estas vulnerabilidades, y ambas merecen una respuesta específica en lugar de un "no se encontraron":

- *Inyección SQL*: la aplicación no construye sentencias SQL. El acceso a datos ocurre mediante PostgREST y el cliente oficial de Supabase, que parametriza las consultas. Las funciones SQL propias no concatenan entrada del usuario: reciben parámetros tipados o leen reclamaciones del JWT.
- XSS: GiroklQ no renderiza HTML. Es una aplicación nativa de SwiftUI y el texto del usuario se dibuja en vistas nativas, no en un motor web.

El riesgo real de esta arquitectura no es la inyección sino la **autorización mal configurada**: una política RLS demasiado permisiva expondría datos de otros usuarios sin necesidad de inyectar nada. Por eso el trabajo de seguridad de esta fase se concentró ahí.

Medidas verificables en el repositorio:

- Row Level Security en todas las tablas de usuario, con políticas `user_id = auth.uid()`.
- `subscription_tier` protegido por un trigger que rechaza cualquier modificación que no provenga de `service_role`.
- Roles imposibles de auto-asignar **por construcción**: la tabla `user_roles` tiene RLS activo y únicamente políticas de `select` para `authenticated`, de modo que insertar, actualizar o borrar el propio rol se rechaza sin depender de una regla que alguien pudiera olvidar.
- Las funciones de administración verifican el rol como primera instrucción y devuelven exclusivamente agregados: conteos, fechas y roles. Nunca contenido de apuntes, títulos, dibujos, historial de chat ni correos electrónicos.
- La decodificación del rol en el cliente falla de forma cerrada: un token malformado, una reclamación ausente o un rol desconocido se resuelven como usuario común.

4. Lecciones aprendidas

- 1. Una configuración de pruebas sin pruebas es peor que no tener ninguna.** El plan de pruebas apuntaba a un target inexistente y deshabilitado. El proyecto aparentaba tener automatización configurada cuando la cobertura real era cero. Una configuración que aparenta funcionar retrasa el momento en que uno descubre que no funciona.
- 2. Medir cobertura sin definir el alcance produce una cifra inútil.** La primera medición sobre el target completo dio 4.7 %. Exigir 80 % sobre ese número habría obligado a escribir pruebas de interfaz de bajo valor únicamente para satisfacer un umbral. Definir explícitamente qué módulos se miden convirtió la métrica en algo accionable, y dejó el 4.7 % como dato honesto de contexto en lugar de ocultarlo.
- 3. Un código de salida cero no significa que algo haya funcionado.** La integración con SonarQube falló en silencio: el scanner subía el reporte y terminaba con éxito aunque el servidor rechazara el análisis por una clave de proyecto incorrecta. El job aparecía en verde mientras nada llegaba al proyecto, y solo al consultar la API de SonarQube se detectó que el último análisis seguía siendo uno anterior. Se corrigió con `sonar.qualitygate.wait`, que obliga a esperar el procesamiento del servidor.
- 4. Programar contra una API supuesta cuesta más que verificarla.** Dos ejecuciones consecutivas del pipeline fallaron porque el script de cobertura asumía la estructura de la salida de `xccov` en lugar de comprobarla. Al descargar un `.xcresult` real y ejecutar el comando se vio que devuelve un diccionario indexado por ruta, no una lista, y además que la consulta por archivo tardaba minutos mientras que una sola consulta global tarda 1.4 segundos. Verificar primero habría evitado ambos errores.
- 5. Un repositorio de código no debe vivir dentro de un servicio de sincronización.** El proyecto estaba en `~/Documents`, sincronizado con iCloud Drive, que genera copias de conflicto con sufijo " 2". Eso costó dos incidentes concretos: una referencia de git corrupta llamada `First-Demo 2` dejó `git fetch` inutilizable durante tres meses, y un duplicado de `TextToolKeyboardBar.swift` rompió la compilación con un error de redeclaración. Se encontraron 24 archivos duplicados, y iCloud recreó uno durante la limpieza. El proyecto se movió a `~/Developer`, fuera de la sincronización.
- 6. Las restricciones de las herramientas gratuitas condicionan el flujo de trabajo, no solo el presupuesto.** El plan gratuito de SonarQube Cloud analiza la rama principal y los pull requests, pero no ramas de trabajo prolongadas. Eso obliga a integrar mediante pull request para obtener métricas, lo cual resulta ser una práctica mejor, pero fue impuesto por la herramienta.
- 7. Los permisos de una credencial son parte de su configuración.** El despliegue falló con `Cloud signing permission error` porque la clave de API de App Store Connect se generó con rol `App Manager`. Subir compilaciones y crear activos de firma son permisos distintos: la firma en la nube requiere rol `Admin`. Regenerar la clave con el rol correcto resolvió el problema de inmediato.
- 8. Dos pipelines desplegando al mismo destino colisionan.** El proyecto tenía Xcode Cloud conectado desde antes, y al añadir el despliegue en GitHub Actions ambos empezaron a subir a TestFlight. App Store Connect rechaza un número de build repetido o menor, y cada sistema lleva su propio contador: el de Actions iba en 22 y el de Xcode Cloud muy por detrás, de modo que este último fallaba en cada push con "the bundle version must be higher than the previously uploaded version". No era un error que corregir sino un pipeline duplicado que retirar. Se desactivó Xcode Cloud, ya que GitHub Actions cubre lo mismo y además ejecuta las pruebas y el umbral de cobertura.

9. Un fallo con un mensaje preciso vale el esfuerzo de escribirlo. El pipeline incluye comprobaciones explícitas: que el secreto exista, que la clave decodificada sea un PEM, que el reporte de cobertura no esté vacío. Esas guardas atraparon un secreto vacío en el segundo paso del despliegue, en lugar de dejar que el proceso compilara diez minutos para morir con un error de firma ilegible. El mensaje nombraba el secreto exacto y el archivo de documentación donde estaba el procedimiento.

5. Plan de mejora continua

5.1 Corto plazo (1 a 2 semanas)

ACCIÓN	RESULTADO ESPERADO
Corregir la generación de 20 preguntas en flashcards	Hoy falla con "la lista de preguntas llegó incompleta", probablemente por truncamiento de la respuesta del modelo
Ampliar la lista de archivos con cobertura exigida conforme se añadan pruebas	Que la cobertura crezca de forma sostenida en lugar de estancarse en los módulos actuales
~~Definir un quality gate propio en SonarQube~~ (completado)	Realizado: gate <code>GirokIQ</code> con condiciones de fiabilidad, seguridad, mantenibilidad, revisión de hotspots y duplicación, todas superadas. La cobertura se controla en el pipeline
Extraer un protocolo de <code>AIService</code> para poder sustituirlo por un doble de prueba	Haría verificable <code>FlashcardsGenerator</code> , hoy imposible de probar porque recibe un cliente concreto
Documentar el procedimiento de aplicación de migraciones	Las migraciones se aplican a mano; el paso se olvida y la aplicación falla pidiendo funciones inexistentes

5.2 Mediano plazo (1 a 2 meses)

ACCIÓN	RESULTADO ESPERADO
Extraer la lógica pura a un módulo o paquete Swift independiente	Medir cobertura por target de forma significativa, sin listas de archivos
Añadir pruebas de integración para <code>SyncEngine</code>	Los dos defectos más costosos del proyecto fueron de sincronización y ninguno era detectable con pruebas unitarias
Automatizar la verificación de las políticas RLS	Una suite que intente leer datos de otro usuario y compruebe que la base de datos los rechaza. Es la mejora de seguridad más valiosa pendiente
Establecer un entorno de Supabase de pruebas y desplegarlo desde el pipeline	Validar que las migraciones aplican limpias desde cero, y dar a ZAP un objetivo que se pueda escanear de forma agresiva
Especificación OpenAPI de las edge functions y <code>zap-api-scan</code> autenticado	Un escaneo pasivo sin autenticación apenas alcanza la superficie real de la API

5.3 Largo plazo

ACCIÓN	RESULTADO ESPERADO
Versión para macOS	Completar el objetivo 6 del plan original
Telemetría de errores en producción	Detectar defectos como el de sincronización sin depender de la observación manual
Generación incremental de flashcards por streaming	Eliminar la dependencia de una única respuesta completa del modelo y mejorar el tiempo de respuesta percibido
Predicción de temas débiles a partir del historial de respuestas	Sugerir repaso dirigido usando los datos que ya se almacenan en <code>flashcards_session</code>
Avanzar hacia CMMI nivel 3	El nivel 2 planteado en la Actividad 4 ya cuenta con evidencia: control de versiones, integración continua, revisión por pull request, despliegue automatizado y registro de cambios

6. Evidencias

ELEMENTO	UBICACIÓN
Código y pipeline	https://github.com/no-c-123/GirokIQ-iOS
Pull request de integración	https://github.com/no-c-123/GirokIQ-iOS/pull/1
Definición del pipeline (pruebas, Sonar y despliegue)	<code>.github/workflows/ci.yml</code>
Escaneo de seguridad	<code>.github/workflows/security-scan.yml</code>
Configuración de SonarQube	<code>sonar-project.properties</code>
Alcance de la cobertura	<code>Scripts/coverage_targets.json</code>
Pruebas unitarias	<code>GirokIQ-iosTests/</code>
Reporte de pruebas	<code>reports/pruebas-unitarias/</code>
Reporte de cobertura	<code>reports/cobertura/</code>
Reporte de calidad	<code>reports/calidad/sonarqube.md</code>
Reporte de seguridad	<code>reports/seguridad/</code>
Diseño de roles	<code>Docs/Roles.md</code>
Procedimiento de despliegue	<code>Docs/Despliegue.md</code>
Migraciones de base de datos	<code>supabase/migrations/</code>